

Longest Substring Without Repeating Characters

Desafio Técnico de Processos Seletivos (LeetCode #3) — Otimização Assintótica de $O(N^3)$ para $O(N)$, Benchmark de CPU e Análise da Janela Deslizante.

Onipresença nos Processos Seletivos

Por que as Big Techs cobram este problema?

Este problema é o divisor de águas entre candidatos que apenas conhecem sintaxe e desenvolvedores que dominam a **otimização de estruturas de dados e complexidade assintótica**.

"Em entrevistas técnicas globais, o código representa apenas 30% da avaliação. Os outros 70% medem clareza de decomposição lógica, comunicação e justificativa de Big-O."

O avaliador busca testar se o candidato percebe a redundância da força bruta e consegue implementar a janela deslizante com consulta em tempo constante $O(1)$.

Empresas com Aplicação Recorrente

Amazon (Top 10 SDE)

Meta (Triagem 45 min)

Google (Nível L3 / L4)

Microsoft

Apple

Bloomberg

Competências Verificadas

- Redução assintótica de $O(N^3)$ para $O(N)$
- Escolha de Hash Map vs. Vetor ASCII estático
- Tratamento de casos de borda e memória de pilha

Enunciado: Substring sem Caracteres Repetidos

Definição do Problema

"Dada uma string s , encontre o comprimento da maior substring contígua que não contenha caracteres repetidos."

Distinção Conceitual Crítica:

- **Substring:** Sequência estritamente contínua na memória (ex: "abc" em "abcde").
- **Subsequência:** Mantém a ordem mas não é contígua (ex: "ace" em "abcde").

Casos de Teste Padronizados

EXEMPLO 1 (GERAL)

$s = \text{"abcabcbb"} \rightarrow$ **Comprimento: 3** (substring: "abc")

EXEMPLO 2 (HOMOGÊNEO)

$s = \text{"bbbbbb"} \rightarrow$ **Comprimento: 1** (substring: "b")

EXEMPLO 3 (COMPOSTO)

$s = \text{"pwwkew"} \rightarrow$ **Comprimento: 3** (substring: "wke")

As 3 Diferentes Abordagens em Código

1. Força Bruta $O(N^3)$

```
def brute_force(s):  
    n = len(s)  
    max_len = 0  
    for i in range(n):  
        for j in range(i+1, n+1):  
            sub = s[i:j]  
            if len(set(sub)) == len(sub):  
                max_len = max(max_len, len(sub))  
    return max_len
```

Tempo: $O(N^3)$

Rejeitado

2. Sliding Window (Set)

```
def sliding_window_set(s):  
    char_set = set()  
    left = max_len = 0  
    for right in range(len(s)):  
        while s[right] in char_set:  
            char_set.remove(s[left])  
            left += 1  
        char_set.add(s[right])  
        max_len = max(max_len, right - left + 1)  
    return max_len
```

Tempo: $O(2N)$

Contratado

3. Sliding Window (Map $O(1)$)

```
def sliding_window_map(s):  
    char_map = {}  
    left = max_len = 0  
    for right, char in enumerate(s):  
        if char in char_map and char_map[char] >= left:  
            left = char_map[char] + 1  
        char_map[char] = right  
        max_len = max(max_len, right - left + 1)  
    return max_len
```

Tempo: $O(N)$ Estrito

Forte Candidato

Código Rodando em Etapas: "abcabcbb"

PASSO	CHAR	R (RIGHT)	L (LEFT)	JANELA	EVENTO / AÇÃO DA TABELA HASH	CHAR_MAP ATUAL	MAX_LEN
1	'a'	0	0	"a"	Primeira ocorrência registrada	{ 'a': 0 }	1
2	'b'	1	0	"ab"	Primeira ocorrência registrada	{ 'a': 0, 'b': 1 }	2
3	'c'	2	0	"abc"	Primeira ocorrência registrada	{ 'a': 0, 'b': 1, 'c': 2 }	3 (Pico Máximo)
4	'a'	3	0 → 1	"bca"	'a' repetido: L salta direto para 0+1=1	{ 'a': 3, 'b': 1, 'c': 2 }	3
5	'b'	4	1 → 2	"cab"	'b' repetido: L salta direto para 1+1=2	{ 'a': 3, 'b': 4, 'c': 2 }	3
6	'c'	5	2 → 3	"abc"	'c' repetido: L salta direto para 2+1=3	{ 'a': 3, 'b': 4, 'c': 5 }	3
7	'b'	6	3 → 5	"cb"	'b' repetido no idx 4: L salta para 4+1=5	{ 'a': 3, 'b': 6, 'c': 5 }	3
8	'b'	7	5 → 7	"b"	'b' repetido no idx 6: L salta para 6+1=7	{ 'a': 3, 'b': 7, 'c': 5 }	3

Economia de CPU: De 4 Horas para 11 Milissegundos

-99,99999%

REDUÇÃO NO TEMPO DE EXECUÇÃO (\$N = 50.000\$)

Redução de 416.000.000× no total de ciclos de instrução de CPU processados.

TAMANHO (N)	FORÇA BRUTA O(N³)	SLIDING WINDOW (MAP O(1))	ECONOMIA
N = 100	4.85 ms (171.700 ops)	0.032 ms (100 ops)	99.94%
N = 1.000	1.953 ms (~2.0 s)	0.232 ms (1.000 ops)	99.99%
N = 5.000	~15.0 s (20.8 bi ops)	1.113 ms (5.000 ops)	99.999%
N = 50.000	~250 min (> 4 HORAS)	11.1 ms (50.000 ops)	> 99.99999%

O Gargalo da Força Bruta

Com \$N = 50.000\$, a Força Bruta executa mais de 20 trilhões de operações, tornando o sistema inoperante em produção.

A Eficiência da Janela Deslizante

A abordagem com Hash Map processa a entrada em passagem única com exatamente 50.000 operações em 0,011 segundos.

Dinâmica dos Dois Ponteiros & Hash Map

SLIDING WINDOW & TWO POINTERS ALGORITHM

VISUALIZING: Longest Substring Without Repeating Characters (LeetCode 3)

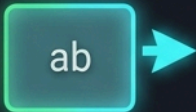
PROBLEM GOAL: Find the length of the longest substring with **NO** repeating characters.

['a' 'b' 'c' 'a' 'b' 'c' 'b' 'b']

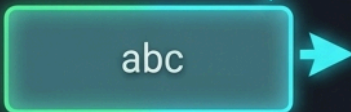


STEP 1: Start pointers L=0, R=0. Max Length = 1.

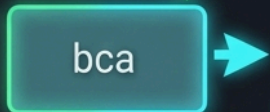
STEP 2: R moves to 'b'.
Window (L=0 to R=1):
Max Length = 2.



STEP 3: R moves to 'c'.
Window (L=0 to R=2):
Max Length = 3.



STEP 4: R moves to 'a' (Duplicate at index 3).
HashMap shows previous 'a' at index 0.
L shifts to index (0+1) = 1.
Window: "bca". Length = 3.



STEP 5: R moves to 'b' (Duplicate at index 4).

CHARACTER lastIndexMap	
Character	Last Seen Index
'a'	0
'b'	1
'c'	2
'a'	3
'b'	4, ...

$L = \max(L, \text{map.get(char_at_R)} + 1)$
when conflict occurs.

Current Max Length:

'a' ['?']

Current Max Length: 3

Mecânica da Otimização

1. Ponteiro Direito (right):

Avança sequencialmente expandindo a janela e lendo novos caracteres.

2. Tabela Hash (char_map):

Armazena o mapeamento `map[char] = ultimo_indice` em tempo constante $O(1)$.

3. Salto do Ponteiro Esquerdo (left):

Ao identificar duplicata, salta diretamente para `map[char] + 1` sem varredura intermediária.

Implementação em C: Tabela Direta ASCII

Código em C com Vetor ASCII Fixo

```
#include <string.h>

int lengthOfLongestSubstring(char* s) {
    int last_index[256];
    for (int i = 0; i < 256; i++) last_index[i] = -1;

    int max_len = 0, left = 0;
    int n = strlen(s);

    for (int right = 0; right < n; right++) {
        unsigned char c = (unsigned char)s[right];

        if (last_index[c] >= left) {
            left = last_index[c] + 1;
        }

        last_index[c] = right;
        int cur_len = right - left + 1;
        if (cur_len > max_len) max_len = cur_len;
    }
    return max_len;
}
```

Vantagens da Implementação em C

- 1. Ausência de Colisões:** Acesso direto por índice do byte ASCII elimina qualquer necessidade de encadeamento ou tratamento de colisão.
- 2. Memória Estática e Restrita:** $256 * \text{sizeof(int)} = 1.024$ bytes (1 KB) alocados diretamente na Stack $\rightarrow$ **espaço $O(1)$ estrito.**
- 3. Eficiência de Cache L1:** Disposição sequencial contígua na memória, sem dispersão de ponteiros na Heap.

Análise Rigorosa de Complexidade (Big-O)

ABORDAGEM	COMPLEXIDADE DE TEMPO	COMPLEXIDADE DE ESPAÇO	CLASSIFICAÇÃO TÉCNICA
1. Força Bruta (Loops Aninhados)	$O(N^3)$	$O(\min(N, \Sigma))$	Rejeitado
2. Sliding Window (Set - Passo a Passo)	$O(2N) = O(N)$	$O(\min(N, \Sigma))$	Contratado
3. Sliding Window (Hash Map - Pulo $O(1)$)	$O(N)$ Estrito	$O(\min(N, \Sigma))$	Forte Candidato

Justificativa de Tempo: $O(N)$

O ponteiro direito avança de 0 até $N-1$ exatamente uma vez. Todas as consultas e inserções no Hash Map executam em tempo constante $O(1)$.

Justificativa de Espaço: $O(\min(N, \Sigma))$

O consumo de memória é limitado pelo menor valor entre o comprimento da string N e a cardinalidade do alfabeto Σ (ASCII = 256).

Casos de Borda & Perguntas da Banca

Casos de Borda Validados

- **String Vazia ""**: Retorna 0 imediatamente.
- **Caracteres Idênticos "bbbbbb"**: A janela se mantém em comprimento 1.
- **Todos Distintos "abcdef"**: A janela expande linearmente até o comprimento total N.
- **Espaços e Símbolos "a b c!"**: Indexados nativamente pela tabela ASCII.

Conclusão

Agradecemos a atenção do Prof. Bianchi e dos colegas.
Abrimos para perguntas e considerações da banca.

[Repositório no GitHub](#)[Vault Obsidian Integrado](#)